

PROYECTO FINAL



Integrantes:

SARA JULIANA PEÑA GÓMEZ

DANIELA FLOREZ VILLAMIL

SANTIAGO RIVEROS

GRUPO: LAS DIVINAS

PROCESAMIENTO DE ALTO VOLUMEN DE DATOS

PONTIFICIA UNIVERSIDAD JAVERIANA

BOGOTÁ D.C

INTRODUCCIÓN

1. FILTROS Y TRANSFORMACIONES

Se tomaron los mismos conjuntos de la primera entrega:

- Pobreza Personas (Departamental)

```
#Dataframe de Personas
urlPersonas = "https://raw.githubusercontent.com/Sarapena02/Procesamiento-de-alto-volumenes-de-datos/refs/heads/main/Conjunto_Datos/Personas%20(%20Departamental%20)%202020.csv"
spark.sparkContext.addFile(urlPersonas)
file_name = "Personas ( Departamental ) 2020.csv"
path1 = SparkFiles.get(file_name)
df_Personas = spark.read.csv("file://" + path1, inferSchema=True, header=True, sep=",")
df_Personas.show(10)
```

(3) Spark Jobs

```
df_Personas: pyspark.sql.dataframe.DataFrame = [DIRECTORIO;SECUENCIA_ENCUESTA;SECUENCIA_P;ORDEN;P6020;P6040;fex_c;P6051;P6090;P5665;P8563;P51;P55;P774;P6160;P8586;P8587;P8587S1;P1088;P1088S1;P6180;P6240;P625;string]
'247300;2;1;2;1;6...'
'247300;3;1;3;1;6...'
'247301;1;1;1;1;8...'
'247301;2;1;2;2;7...'
'247302;1;1;1;1;5...'
'247302;2;1;2;2;5...'
'247302;3;1;3;1;2...'
'247302;4;1;4;1;2...'
'247302;5;1;5;1;7...'
-----
```

- Internet Fijo por Penetración por Municipios

```
#Dataframe Internet
urlInternet = "https://raw.githubusercontent.com/Sarapena02/Procesamiento-de-alto-volumenes-de-datos/refs/heads/main/Conjunto_Datos/Internet_Fijo_Penetraci_n_Municipio_20240828.csv"
spark.sparkContext.addFile(urlInternet)
file_name = "Internet_Fijo_Penetraci_n_Municipio_20240828.csv"
path2 = SparkFiles.get(file_name)
df_Internet = spark.read.csv("file://" + path2, inferSchema=True, header=True, sep=",")
df_Internet.show(10)
```

(3) Spark Jobs

```
df_Internet: pyspark.sql.dataframe.DataFrame = [AÑO: integer, TRIMESTRE: integer ... 7 more fields]
```

AÑO	TRIMESTRE	COD_DEPARTAMENTO	DEPARTAMENTO	COD_MUNICIPIO	MUNICIPIO	No. ACCESOS FIJOS A INTERNET	POBLACIÓN DANE	INDICE
2023	2	5	ANTIOQUIA	5353	HISPANIA	495	5790	8,55
2023	1	17	CALDAS	17001	MANIZALES	102906	454494	22,64
2023	1	52	NARIÑO	52356	IPIALES	11312	120842	9,36
2022	4	27	CHOCÓ	27160	CÉRTEGUI	179	5967	3,00
2022	3	15	BOYACÁ	15740	SIACHOQUE	116	7056	1,64
2022	2	13	BOLÍVAR	13580	REGIDOR	77	7421	1,04
2022	2	5	ANTIOQUIA	5091	BETANIA	822	10718	7,67
2022	1	25	CUNDINAMARCA	25524	PANDI	405	5587	7,25
2022	2	5	ANTIOQUIA	5172	CHIGORODÓ	9896	61211	16,17
2022	3	85	CASANARE	85136	LA SALINA	26	1388	1,87

only showing top 10 rows

- Educación en Preescolar, Primaria, Básica y Media por Municipio

```
#DataFrame Educación
urlEducacion = "https://raw.githubusercontent.com/Sarapena02/Procesamiento-de-alto-volumenes-de-datos/refs/heads/main/Conjunto_Datos/MEN_ESTADISTICAS_EN_EDUCACION_EN_PREESCOLAR_B_SICA_Y_MEDIA_POR_MUNICIPIO_20240829.csv"
spark.sparkContext.addFile(urlEducacion)
file_name = "MEN_ESTADISTICAS_EN_EDUCACION_EN_PREESCOLAR_B_SICA_Y_MEDIA_POR_MUNICIPIO_20240829.csv"
path3 = SparkFiles.get(file_name)
df_Educacion = spark.read.csv("file://" + path3, inferSchema=True, header=True, sep=",")
df_Educacion.show(10)
```

3) Spark Jobs

```
df_Educacion: pyspark.sql.dataframe.DataFrame = [AÑO: string, CÓDIGO_MUNICIPIO: integer ... 39 more fields]
68| 95.46| 96.94| 95.12| 95.61| 95.63| 0.58| 0.0|
04| 2.69| 5.41| 0.73| 5.53| 6.9| 4.11|
011| 5036|Angelópolis| 5| Antioquia| 3758|Antioquia (ETC)| 1707| 78.85|
.2| 82.4| 59.9| 26.4| 82.8| 77.4|
.4| 40.1| 19.57| 100.0| 8.14| 6.6| 6.1|
44| 86.27| 93.4| 90.29| 77.32| 90.24| 5.59| 0.0|
5| 7.32| 0.71| 0.0| 0.7| 1.08| 0.0|
011| 5038| Angostura| 5| Antioquia| 3758|Antioquia (ETC)| 3344| 81.7|
.5| 92.6| 49.6| 20.3| 90.8| 72.1|
.8| 50.1| 20.29| 86.49| 5.32| 6.45| 4.79|
64| 91.85| 93.55| 91.49| 92.85| 89.09| 2.83| 0.0|
93| 2.27| 3.92| 0.0| 5.16| 1.92| 2.96|
011| 5040| Anorí| 5| Antioquia| 3758|Antioquia (ETC)| 3983| 92.09|
.9| 110.4| 51.1| 18.1| 100.7| 85.7|
.3| 44.8| 20.09| 63.93| 6.26| 4.63| 8.33|
81| 92.16| 95.37| 89.53| 96.52| 95.76| 1.58| 0.0|
0| 0.42| 0.48| 0.0| 0.69| 0.0| 0.0|
```

- ICFES Saber 11 2019

```
#DataFrame Saber ICFES
df_Saber = spark.read.format("csv").option("header", "true").load("dbfs:/FileStore/shared_uploads/sarapenagomez02@gmail.com/Saber_11_2019_2_20240829-3.csv")
df_Saber.show(10)
```

(2) Spark Jobs

```
df_Saber: pyspark.sql.dataframe.DataFrame = [ESTU_TIPODOCUMENTO: string, ESTU_NACIONALIDAD: string ... 80 more fields]
> Si| No| No| 11 A 25 LIBROS| Todos o casi todo...| 3 a 5 veces por s...| 3 a
veces por s...| Igual| Entre 30 y 60 min...| Entre 30 y 60 min...| 0| No| 1796|
370001039395| CENTRO EDUCATIVO ...| MIXTO| NO OFICIAL| A| -| ACADÉMICO| 370001039395|
ITRO EDUCATIVO ...| S| URBANO| MAÑANA| 70001| SINCELEJO| 70|
JCRC| N| 70001| SINCELEJO| SUCRE| 70| 68|
4| 66| 91| 3| 63| 91| 3|
100| 4| 51| 61| A1| 336| 95| 58.8176527963198|
3| PUBLICAR| NO|
```

Para poder realizar los filtros y transformaciones necesarias para responder los siguientes puntos del proyecto.

Se inicio con el Dataframe de Personas. El cual al cargar se vio que las columnas no estaban separadas y Databricks estaba tomando todas las variables como una sola. Por ende, se realizo un ciclo para separar cada una de las variables:

- ▶ (1) Spark Jobs

[illegible]

Luego de separar las columnas, se cambia el nombre de la variable de la columna “P6020” por “Sexo” para identificar la variable al hacer el análisis correspondiente

- ▶ (1) Spark Jobs

ENCUESTA DE SEGURIDAD Y SALUD EN EL TRABAJO																
ENCUESTA DE SEGURIDAD Y SALUD EN EL TRABAJO																
DIRECTORIO SECUENCIA_ENCUESTA SECUENCIA_P ORDEN Sexo P6040 fex_c P6051 P6090 P5655 P8563 P51 P55 P774 P6160 P8586 P8587 P858751 P1088 P108851 P6180 P6240 P6250 P6260 P6270 P6351 P6390 P7250 P6920																

```
#Se encuentran los valores nulos del dataframe
df_Personas2.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in df_Personas2.columns]).show()
```

- ▶ (2) Spark Jobs

[illegible]

Al no estar tan claro cuales variables presentan valores nulos, se escogieron las variables adecuadas para realizar análisis futuros:

```
df_Personas3 = df_Personas1.select(
    df_Personas1["P6020"].alias("Sexo"),
    df_Personas1["P6040"],
    df_Personas1["P6160"],
    df_Personas1["P8587"]
)
df_Personas3.show()
```

▶ (1) Spark Jobs

▶ df_Personas3: pyspark.sql.dataframe.DataFrame

Sexo	P6040	P6160	P8587
2	71	null	null
1	67	null	null
1	64	null	null
1	84	null	null
2	76	null	null
1	59	null	null
2	56	null	null

Sin embargo, dado las preguntas del negocio y como esta estructurado este Dataframe, se decidió no usarlo y guardarlo para posteriores análisis.

El siguiente Dataframe, es el de Penetración de Internet por Municipio.

Lo primero que se realiza es el cambio de nombre de las variables, ya que algunas contaban con espacios o con caracteres especiales lo que podía afectar el análisis que se realizó posteriormente:

```
3 days ago (<1s) 16

# Se hara el cambio de nombre a las variables
nuevasColumnas = ['ANIO', 'TRIMESTRE', 'COD_DEPT', 'DEPARTAMENTO', 'COD_MUN', 'MUNICIPIO', 'No_ACCESOS_FIJOS_A_INTERNET', 'POBLACION_DANE', 'INDICE']
df_Internet1 = df_Internet

for antes, despues in zip (df_Internet1.columns, nuevasColumnas):
    df_Internet1 = df_Internet1.withColumnRenamed(antes,despues)

df_Internet1.columns

▶ df_Internet1: pyspark.sql.dataframe.DataFrame = [ANIO: integer, TRIMESTRE: integer ... 7 more fields]
Out[11]: ['ANIO',
'TRIMESTRE',
'COD_DEPT',
'DEPARTAMENTO',
'COD_MUN',
'MUNICIPIO',
'No_ACCESOS_FIJOS_A_INTERNET',
'POBLACION_DANE',
'INDICE']
```

Se procede a hacer una selección de variables que servirán para realizar el análisis para dar respuesta a las preguntas del negocio.

```
#Se seleccionan las siguientes variables: "AÑO", "DEPARTAMENTO", "MUNICIPIO", "No_ACCESOS_FIJOS_A_INTERNET" y "POBLACION_DANE"
df_Internet2 = df_Internet1.select(
    df_Internet1["AÑO"],
    df_Internet1["DEPARTAMENTO"],
    df_Internet1["MUNICIPIO"],
    df_Internet1["No_ACCESOS_FIJOS_A_INTERNET"],
    df_Internet1["POBLACION_DANE"]
)

# Mostrar el DataFrame resultante
df_Internet2.show()
```

(1) Spark Jobs

df_Internet2: pyspark.sql.dataframe.DataFrame = [AÑO: integer, DEPARTAMENTO: string ... 3 more fields]

AÑO	DEPARTAMENTO	MUNICIPIO	No_ACCESOS_FIJOS_A_INTERNET	POBLACION_DANE
2023	ANTIOQUIA	HISPANIA	495	5790
2023	CALDAS	MANIZALES	102906	454494
2023	NARIÑO	IPIALES	11312	120842
2022	CHOCÓ	CÉRTEGUI	179	5967
2022	BOYACÁ	SIACHOQUE	116	7056
2022	BOLÍVAR	REGIDOR	77	7421
2022	ANTIOQUIA	BETANIA	822	10718
2022	CUNDINAMARCA	PANDI	405	5587
2022	ANTIOQUIA	CHIGORODÓ	9896	61211
2022	CASANARE	LA SALINA	26	1388
2021	BOLÍVAR	CALAMAR	564	24034
2021	CESAR	AGUACHICA	11927	121581
2022	BOYACÁ	SUSACÓN	8	2798
2023	CHOCÓ	ATRATO	20	6724
2022	NORTE DE SANTANDER	SARDINATA	704	28397
2021	NARIÑO	OLAYA HERRERA	94	25969
2021	VICHADA	SANTA ROSALÍA	9	4367
2023	BOYACÁ	CERTINZA	233	3012

Se revisa si las variables seleccionadas presentan valores nulos para realizar el respectivo tratamiento:

```
#Se revisan los valores nulos de las variables seleccionadas
df_Internet2.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in df_Internet2.columns]).show()
```

(2) Spark Jobs

AÑO	DEPARTAMENTO	MUNICIPIO	No_ACCESOS_FIJOS_A_INTERNET	POBLACION_DANE
0	0	0	0	0

También se revisa si tipo de dato de las variables corresponde a la información que están almacenando:

```
#Se revisa el tipo de datos de las variables
df_Internet3.printSchema()

root
 |-- AÑO: integer (nullable = true)
 |-- DEPARTAMENTO: string (nullable = true)
 |-- MUNICIPIO: string (nullable = true)
 |-- No_ACCESOS_FIJOS_A_INTERNET: integer (nullable = true)
 |-- POBLACION_DANE: integer (nullable = true)
```

Como no se encontró valores inconsistentes y los tipos de datos corresponden a las variables, se pasa a la selección de departamentos dentro del conjunto de datos para filtrar la información del Dataframe:

```

# Se realiza un filtrado para obtener solo los datos de los departamentos "NARIÑO" y "MAGDALENA"
df_Internet3 = df_Internet2.filter(
    (df_Internet2["DEPARTAMENTO"] == "NARIÑO") |
    (df_Internet2["DEPARTAMENTO"] == "MAGDALENA")
)

# Mostrar el DataFrame filtrado
df_Internet3.show()

```

► (1) Spark Jobs

► df_Internet3: pyspark.sql.dataframe.DataFrame = [ANIO: integer, DEPARTAMENTO: string ... 3 more fields]

ANIO	DEPARTAMENTO	MUNICIPIO	No_ACCESOS_FIJOS_A_INTERNET	POBLACION_DANE
2023	NARIÑO	IPIALES	11312	120842
2021	NARIÑO	OLAYA HERRERA	94	25969
2022	NARIÑO	LA UNIÓN	451	33028
2023	NARIÑO	PASTO	87688	410835
2023	NARIÑO	SANDONÁ	680	21244
2022	MAGDALENA	CIÉNAGA	11067	130000
2022	NARIÑO	CHACHAGÜÍ	1877	15769
2022	NARIÑO	LA UNIÓN	399	33028
2022	NARIÑO	BUESACO	941	24726
2022	MAGDALENA	EL BANCO	1776	72286
2023	NARIÑO	POLICARPA	968	10314
2021	NARIÑO	GUACHUCAL	309	20089
2021	MAGDALENA	TENERIFE	822	14271
2021	NARIÑO	ALDANA	23	7636
2021	MAGDALENA	TENERIFE	826	14271
2021	MAGDALENA	SALAMINA	1037	11657
2021	NARIÑO	PUPIALES	265	17946
2022	MAGDALENA	CERRO SAN ANTONIO	6	10954

Por último, se especifica los años que se van a tomar para el análisis de las preguntas del negocio. Se decidió que se tomará desde el 2019 para que sea consistente con la información del conjunto de datos que se tomó del ICFES Saber 11 del 2019.

```
#Se especifican solo los años que se tienen en cuenta para el analisis.
#Se toma desde el 2019 por las pruebas ICFES
df_Internet3 = df_Internet3.filter(df_Internet3["ANIO"]>= 2019)
df_Internet3.show()
```

▶ (1) Spark Jobs

▶ df_Internet3: pyspark.sql.dataframe.DataFrame = [ANIO: integer, DEPARTAMENTO: string ... 3 mo

ANIO	DEPARTAMENTO	MUNICIPIO	No_ACCESOS_FIJOS_A_INTERNET	POBLACION_DANE
2023	NARIÑO	IPIALES	11312	120842
2021	NARIÑO	OLAYA HERRERA	94	25969
2022	NARIÑO	LA UNIÓN	451	33028
2023	NARIÑO	PASTO	87688	410835
2023	NARIÑO	SANDONÁ	680	21244
2022	MAGDALENA	CIÉNAGA	11067	130000
2022	NARIÑO	CHACHAGÜÍ	1877	15769
2022	NARIÑO	LA UNIÓN	399	33028
2022	NARIÑO	BUESACO	941	24726
2022	MAGDALENA	EL BANCO	1776	72286
2023	NARIÑO	POLICARPA	968	10314
2021	NARIÑO	GUACHUCAL	309	20089
2021	MAGDALENA	TENERIFE	822	14271
2021	NARIÑO	ALDANA	23	7636
2021	MAGDALENA	TENERIFE	826	14271
2021	MAGDALENA	SALAMINA	1037	11657
2021	NARIÑO	PUPIALES	265	17946
2022	MAGDALENA	CERRO SAN ANTONIO	6	10954

Se realiza un procedimiento similar con el DataFrame de Educación en Preescolar, Primaria, Básica y Media Secundaria:

```
#Se cambian de nombres las variables de las columnas para tener un mejor manejo de este DF:

nuevasColumnas = ['ANIO', 'COD_MUN', 'MUNICIPIO', 'COD_DEPT', 'DEPARTAMENTO', 'COD_ETC', 'ETC', 'POBLACION_5_16', 'TASA_M_5_16', 'COBERTURA_NETA', 'COBERTURA_N_T', 'COBERTURA_N_P',
'COBERTURA_N_S', 'COBERTURA_N_M', 'COBERTURA_BRUTA', 'COBERTURA_B_T', 'COBERTURA_B_P', 'COBERTURA_B_S', 'COBERTURA_B_M', 'TAM_PROM_GRUPO', 'SEDES_CON_INTERNET', 'DESERCIÓN', 'DESERCIÓN_T',
'DESERCIÓN_P', 'DESERCIÓN_S', 'DESERCIÓN_M', 'APROBACIÓN', 'APROBACIÓN_T', 'APROBACIÓN_P', 'APROBACIÓN_S', 'APROBACIÓN_M', 'REPROBACIÓN', 'REPROBACIÓN_T', 'REPROBACIÓN_P', 'REPROBACIÓN_S',
'REPROBACIÓN_M', 'REPITIENCIA', 'REPITIENCIA_T', 'REPITIENCIA_P', 'REPITIENCIA_S', 'REPITIENCIA_M']
df_Educacion1 = df_Educacion

for antes, despues in zip(df_Educacion1.columns, nuevasColumnas):
    df_Educacion1 = df_Educacion1.withColumnRenamed(antes, despues)

df_Educacion1.columns
```

df_Educacion1: pyspark.sql.dataframe.DataFrame = [ANIO: string, COD_MUN: integer ... 39 more fields]

SEDES_CON_INTERNET',
DESERCIÓN',
DESERCIÓN_T',
DESERCIÓN_P',
DESERCIÓN_S',
DESERCIÓN_M',
APROBACIÓN',
APROBACIÓN_T',
APROBACIÓN_P',
APROBACIÓN_S',
APROBACIÓN_M',
REPROBACIÓN',
REPROBACIÓN_T',
REPROBACIÓN_P',
REPROBACIÓN_S',
REPROBACIÓN_M',
REPITIENCIA',
REPITIENCIA_T',
REPITIENCIA_P',
REPITIENCIA_S',
REPITIENCIA_M']

Se hace selección de las variables necesaria para el análisis:


```
#Se seleccionan las siguientes variables: "ANIO", "COD_DEPT", "DEPARTAMENTO", "MUNICIPIO", "TASA_M_5_16", "COBERTURA_NETA" y "SEDES_CON_INTERNET"
df_Educacion2 = df_Educacion1.select(
    df_Educacion1["ANIO"],
    df_Educacion1["MUNICIPIO"],
    df_Educacion1["COD_DEPT"],
    df_Educacion1["DEPARTAMENTO"],
    df_Educacion1["TASA_M_5_16"],
    df_Educacion1["COBERTURA_NETA"],
    df_Educacion1["SEDES_CON_INTERNET"]
)

# Mostrar el DataFrame resultante
df_Educacion2.show()
```

▶ (1) Spark Jobs

df_Educacion2: pyspark.sql.dataframe.DataFrame = [ANIO: string, MUNICIPIO: string ... 5 more fields]

[2011]	Abriaquí	5	Antioquia	88.61	88.6	75.0
[2011]	Alejandro	5	Antioquia	118.52	118.5	81.25
[2011]	Amagá	5	Antioquia	78.65	78.7	83.33
[2011]	Amalfi	5	Antioquia	97.71	97.7	53.45
[2011]	Andes	5	Antioquia	84.45	84.5	93.44
[2011]	Angelópolis	5	Antioquia	78.85	78.9	100.0
[2011]	Angostura	5	Antioquia	81.7	81.7	86.49
[2011]	Anorí	5	Antioquia	92.09	92.1	63.93
[2011]	Santafé de Antioquia	5	Antioquia	84.59	84.6	90.91
[2011]	Anzá	5	Antioquia	86.07	86.1	88.89
[2011]	Apartadó	5	Antioquia	66.51	66.5	65.91
[2011]	Arboletes	5	Antioquia	84.85	84.9	47.22
[2011]	Argelia	5	Antioquia	88.58	88.6	82.98
[2011]	Armenia	5	Antioquia	87.97	88.0	100.0
[2011]	Barbosa	5	Antioquia	82.05	82.1	84.0
[2011]	Belmira	5	Antioquia	97.76	97.8	100.0
[2011]	Bello	5	Antioquia	82.53	82.4	87.5
[2011]	Betania	5	Antioquia	89.85	89.9	96.3

only showing top 20 rows

Se revisan los valores nulos de las variables seleccionadas:

```
#Se revisan los valores nulos de las variables seleccionadas
df_Educacion2.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in df_Educacion2.columns]).show()
```

▶ (2) Spark Jobs

ANIO	MUNICIPIO	COD_DEPT	DEPARTAMENTO	TASA_M_5_16	COBERTURA_NETA	SEDES_CON_INTERNET
0	0	0	0	115	111	5696

Como se puede observar, tres variables (“TASA_M_5_16”, “SEDES_CON_INTERNET”, “COBERTURA_NETA”) cuentan con valores nulos por lo que se realizó el tratamiento adecuado. Consistió en cambiar esos valores nulos por el promedio de cada columna:

```
# Rellenamos 'TASA_M_5_16', 'COBERTURA_NETA' y 'SEDES_CON_INTERNET' con el promedio
media_tasa_Educacion = df_Educacion2.select(avg("TASA_M_5_16")).first()[0]
df_Educacion2 = df_Educacion2.fillna({"TASA_M_5_16": media_tasa_Educacion})

# Cobertura
media_tasa_Cobertura = df_Educacion2.select(avg("COBERTURA_NETA")).first()[0]
df_Educacion2 = df_Educacion2.fillna({"COBERTURA_NETA": media_tasa_Educacion})

# Sedes
media_tasa_Sedes = df_Educacion2.select(avg("SEDES_CON_INTERNET")).first()[0]
df_Educacion2 = df_Educacion2.fillna({"SEDES_CON_INTERNET": media_tasa_Educacion})

#Se revisan los valores nulos de las variables seleccionadas
df_Educacion2.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in df_Educacion2.columns]).show()
```

▶ (8) Spark Jobs

df_Educacion2: pyspark.sql.dataframe.DataFrame = [ANIO: string, MUNICIPIO: string ... 5 more fields]

ANIO	MUNICIPIO	COD_DEPT	DEPARTAMENTO	TASA_M_5_16	COBERTURA_NETA	SEDES_CON_INTERNET
0	0	0	0	0	0	0

De esta forma, el manejo y el análisis de este DataFrame será más consistente y real.

Adicionalmente, se confirma que los tipos de datos son consistentes con la información que las variables están almacenando:

```
#Se revisa el tipo de dato de las variables escogidas
df_Educacion3.printSchema()

root
 |-- ANIO: string (nullable = true)
 |-- MUNICIPIO: string (nullable = true)
 |-- COD_DEPT: integer (nullable = true)
 |-- DEPARTAMENTO: string (nullable = true)
 |-- TASA_M_5_16: double (nullable = false)
 |-- COBERTURA_NETA: double (nullable = false)
 |-- SEDES_CON_INTERNET: double (nullable = false)
```

Se procede a seleccionar los departamentos que se van a trabajar. En este caso, fue necesario identificarlos por medio de sus códigos ya que el Dataframe no encontraba la información por medio de los nombres de estos.

```
# Se realiza un filtrado para obtener solo los datos de los departamentos "NARIÑO" y "MAGDALENA"
df_Educacion3 = df_Educacion2.filter(
    (df_Educacion2["COD_DEPT"] == "47") | #Magdalena
    (df_Educacion2["COD_DEPT"] == "52") # Nariño
)

# Mostrar el DataFrame filtrado
df_Educacion3.show()
```

▶ (1) Spark Jobs

▶ df_Educacion3: pyspark.sql.dataframe.DataFrame = [ANIO: string, MUNICIPIO: string ... 5 more fields]

ANIO	MUNICIPIO	COD_DEPT	DEPARTAMENTO	TASA_M_5_16	COBERTURA_NETA	SEDES_CON_INTERNET
2011	Francisco Pizarro	52	Nariño	49.68	49.7	11.76
2011	Santa Marta	47	Magdalena	86.98	86.7	23.23
2011	Algarrobo	47	Magdalena	104.08	104.1	35.71
2011	Aracataca	47	Magdalena	83.37	83.4	25.71
2011	Ariguani	47	Magdalena	106.8	106.8	22.86
2011	Cerro San Antonio	47	Magdalena	93.61	93.6	40.0
2011	Chibolo	47	Magdalena	90.0	90.0	58.33
2011	Ciénaga	47	Magdalena	94.79	94.6	22.22
2011	Concordia	47	Magdalena	130.38	130.4	50.0
2011	El Banco	47	Magdalena	101.79	101.8	28.0
2011	El Piñon	47	Magdalena	93.07	93.1	31.58
2011	El Retén	47	Magdalena	108.0	108.0	75.0
2011	Fundación	47	Magdalena	109.21	108.9	41.27
2011	Guamal	47	Magdalena	100.08	100.1	13.33
2011	Nueva Granada	47	Magdalena	103.54	103.3	4.17
2011	Pedraza	47	Magdalena	105.38	105.4	20.0
2011	Pijiño del Carmen	47	Magdalena	76.77	76.8	13.33
2011	Piñon	47	Magdalena	104.79	104.8	35.29

Por último, se realizó un filtro del año desde el 2019 para que sea consistente con los datos proporcionados del ICFES.

```

#Se especifican solo los años que se tienen en cuenta para el analisis.
#Se toma desde el 2019 por las pruebas ICFES
df_Educacion3 = df_Educacion3.filter(df_Educacion3["ANIO"]>=2019)
df_Educacion3.show()

```

▶ (1) Spark Jobs

▶ df_Educacion3: pyspark.sql.dataframe.DataFrame = [ANIO: string, MUNICIPIO: string ... 5 more fields]

ANIO	MUNICIPIO	COD_DEPT	DEPARTAMENTO	TASA_M_5_16	COBERTURA_NETA	SEDES_CON_INTERNET
2019	Roberto Payán	52	Nariño	87.99	87.99	85.08948968462063
2019	Santa Marta	47	Magdalena	102.9	102.57	85.08948968462063
2019	Algarrobo	47	Magdalena	89.88	89.61	85.08948968462063
2019	Aracataca	47	Magdalena	81.7	81.5	85.08948968462063
2019	Ariguaní	47	Magdalena	97.1	96.98	85.08948968462063
2019	Cerro San Antonio	47	Magdalena	87.84	87.84	85.08948968462063
2019	Chibolo	47	Magdalena	88.34	88.34	85.08948968462063
2019	Ciénaga	47	Magdalena	94.25	93.87	85.08948968462063
2019	Concordia	47	Magdalena	102.48	102.48	85.08948968462063
2019	El Banco	47	Magdalena	100.58	100.53	85.08948968462063
2019	El Piñon	47	Magdalena	73.69	73.69	85.08948968462063
2019	El Retén	47	Magdalena	106.65	106.65	85.08948968462063
2019	Fundación	47	Magdalena	101.61	101.07	85.08948968462063
2019	Guamal	47	Magdalena	101.01	101.01	85.08948968462063
2019	Nueva Granada	47	Magdalena	93.19	93.17	85.08948968462063
2019	Pedraza	47	Magdalena	95.3	95.3	85.08948968462063
2019	Pijiño del Carmen	47	Magdalena	111.73	111.73	85.08948968462063
2019	Piviaví	47	Magdalena	98.83	98.79	85.08948968462063

El último Dataframe seleccionado es el del ICFES Saber 11 2019. Se realizó un procedimiento similar a los conjuntos de datos anteriores.

Se cambian el nombre de las variables del Dataframe:

```

"EDUCACION_PAURE", "EDUCACION_MADRE", "TRABAJOLABOR_PAURE", "TRABAJOLABOR_MADRE",
"TIENE_INTERNET", "TIENE_SERVICIOTV", "TIENE_COMPUTADOR", "TIENE_LAVADORA",
"TIENE_HORNO_MICROOGAS", "TIENE_AUTOMOVIL", "TIENE_MOTOCICLETA", "TIENE_CONSOLA_VIDEOJUEGOS",
"NUM_LIBROS", "COME_LECHE_DERIVADOS", "COME_CARNE_PESCADO_HUEVO", "COME_CEREAL_FRUTOS_LEGUMBRE",
"SITUACION_ECONOMICA", "DEDICACION_LECTURA_DIARIA", "DEDICACION_INTERNET",
"HORAS_SEMANA_TRABAJA", "TIPO_REMUNERACION", "CODIGO_ICFES", "COD_DANE_ESTABLECIMIENTO",
"NOMBRE_ESTABLECIMIENTO", "GENERO_ESTABLECIMIENTO", "NATURALEZA_ESTABLECIMIENTO",
"CALENDARIO_ESTABLECIMIENTO", "BILINGUE_ESTABLECIMIENTO", "CARACTER_ESTABLECIMIENTO",
"COD_DANE_SEDE", "NOMBRE_SEDE", "SEDE_PRINCIPAL", "AREA_UBICACION", "JORNADA",
"COD_MCPIO_UBICACION", "MCPIO_UBICACION", "COD_DEPTO_UBICACION", "DEPTO_UBICACION",
"PRIVADO_LIBERTAD", "COD_MCPIO_PRESENTACION", "MCPIO_PRESENTACION", "DEPTO_PRESENTACION",
"COD_DEPTO_PRESENTACION", "PUNT_LECTURA_CRITICA", "PERCENTIL_LECTURA_CRITICA",
"DESEMP_LECTURA_CRITICA", "PUNT_MATEMATICAS", "PERCENTIL_MATEMATICAS", "DESEMP_MATEMATICAS",
"PUNT_C_NATURALES", "PERCENTIL_C_NATURALES", "DESEMP_C_NATURALES", "PUNT_SOCIALES_CIUDADANAS",
"PERCENTIL_SOCIALES_CIUDADANAS", "DESEMP_SOCIALES_CIUDADANAS", "PUNT_INGLES",
"PERCENTIL_INGLES", "DESEMP_INGLES", "PUNT_GLOBAL", "PERCENTIL_GLOBAL", "INSE_INDIVIDUAL",
"NSE_INDIVIDUAL", "NSE_ESTABLECIMIENTO", "ESTADODINVESTIGACION", "GENERACION_E"
}

df_Saber1 = df_Saber

# Renombrar cada columna usando un ciclo
for antes, despues in zip(df_Saber1.columns, nuevasColumnas_Saber):
    df_Saber1 = df_Saber1.withColumnRenamed(antes, despues)

# Ver las columnas del DataFrame final
df_Saber1.columns

df_Saber1: pyspark.sql.dataframe.DataFrame = [TIPODOCUMENTO: string, NACIONALIDAD: string ... 80 more fields]
'PERCENTIL_LECTURA_CRITICA',
'DESEMP_LECTURA_CRITICA',
'PUNT_MATEMATICAS',
'PERCENTIL_MATEMATICAS',
'DESEMP_MATEMATICAS',
'PUNT_C_NATURALES',
'PERCENTIL_C_NATURALES',
'DESEMP_C_NATURALES',
'PUNT_SOCIALES_CIUDADANAS',
'PERCENTIL_SOCIALES_CIUDADANAS',
'DESEMP_SOCIALES_CIUDADANAS',
'PUNT_INGLES',
'PERCENTIL_INGLES',
'DESEMP_INGLES',
'PUNT_GLOBAL',
'PERCENTIL_GLOBAL',
'INSE_INDIVIDUAL',
'NSE_INDIVIDUAL',
'NSE_ESTABLECIMIENTO',
'ESTADODINVESTIGACION',
'GENERACION_E'

```

Se seleccionan las variables correspondientes para proceder con el análisis:

```
#Se seleccionan las siguientes variables: "PERIODO", "COD_RESIDE_DEPTO", "DEPTO_RESIDE", "MCPIO_RESIDE", "TIENE_INTERNET", "PUNT_GLOBAL"
df_Saber2 = df_Saber1.select(
    df_Saber1["PERIODO"],
    df_Saber1["COD_RESIDE_DEPTO"],
    df_Saber1["DEPTO_RESIDE"],
    df_Saber1["MCPIO_RESIDE"],
    df_Saber1["TIENE_INTERNET"],
    df_Saber1["PUNT_GLOBAL"]
)

# Mostrar el DataFrame resultante
df_Saber2.show()
```

► (1) Spark Jobs

df_Saber2: pyspark.sql.dataframe.DataFrame = [PERIODO: string, COD_RESIDE_DEPTO: string ... 4 more fields]

PERIODO	COD_RESIDE_DEPTO	DEPTO_RESIDE	MCPIO_RESIDE	TIENE_INTERNET	PUNT_GLOBAL
20194	47	MAGDALENA	SANTA ANA	Sí	208
20194	11	BOGOTÁ	BOGOTÁ D.C.	Sí	299
20194	13	BOLIVAR	CARTAGENA DE INDIAS	No	299
20194	11	BOGOTÁ	BOGOTÁ D.C.	No	309
20194	11	BOGOTÁ	BOGOTÁ D.C.	Sí	288
20194	08	ATLANTICO	SOLEDAD	-	198
20194	76	VALLE	CALI	Sí	355
20194	68	SANTANDER	BUCARAMANGA	Sí	313
20194	25	CUNDINAMARCA	SOACHA	Sí	288
20194	70	SUCRE	SINCELEJO	Sí	336
20194	13	BOLIVAR	MONTECRISTO	No	187
20194	47	MAGDALENA	CONCORDIA	Sí	166
20194	11	BOGOTÁ	BOGOTÁ D.C.	Sí	302
20194	11	BOGOTÁ	BOGOTÁ D.C.	Sí	281
20194	05	ANTIOQUIA	MEDELLÍN	No	220
20194	20	CESAR	EL PASO	Sí	269
20194	15	BOYACA	UITAMA	Sí	319
20194	41	HUILA	NEIVA	Sí	263

Se realiza una inspección de los datos nulos de las variables seleccionadas:

```
#Se revisan los valores nulos de las variables seleccionadas
df_Saber2.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in df_Saber2.columns]).show()
```

► (2) Spark Jobs

PERIODO	COD_RESIDE_DEPTO	DEPTO_RESIDE	MCPIO_RESIDE	TIENE_INTERNET	PUNT_GLOBAL
0	0	377	377	8337	0

Se observa que tres variables cuentan con valores nulos y se procede a realizar un procedimiento similar al conjunto de datos anterior:

```

# Especificar los valores por defecto solo para las columnas necesarias
valores_por_defecto = {
    "DEPTO_RESIDE": "Desconocido",
    "MCPIO_RESIDE": "Desconocido",
    "TIENE_INTERNET": "No especificado"
}

# Rellenar valores nulos en las columnas específicas
df_Saber2 = df_Saber2.fillna(valores_por_defecto, subset=["DEPTO_RESIDE", "MCPIO_RESIDE", "TIENE_INTERNET"])

#Se revisan los valores nulos de las variables seleccionadas
df_Saber2.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in df_Saber2.columns]).show()

▶ (2) Spark Jobs

▶ df_Saber2: pyspark.sql.dataframe.DataFrame = [PERIODO: string, COD_RESIDE_DEPTO: string ... 4 more fields]

+-----+-----+-----+-----+-----+-----+
|PERIODO|COD_RESIDE_DEPTO|DEPTO_RESIDE|MCPIO_RESIDE|TIENE_INTERNET|PUNT_GLOBAL|
+-----+-----+-----+-----+-----+-----+
|      0|              0|           0|           0|             0|          0|
+-----+-----+-----+-----+-----+-----+

```

Al no ser variables categóricas, se procede a que los datos que hacen falta se cambien de caracteres especiales a frases para tener un mejor manejo del conjunto de datos. De esta forma, se evita que en el análisis den datos o información falsa.

Se procede a seleccionar los departamentos de enfoque para seguir con el tratamiento y análisis del conjunto de datos:

```

# Se realiza un filtrado para obtener solo los datos de los departamentos "NARIÑO" y "MAGDALENA"
df_Saber3 = df_Saber2.filter(
    (df_Saber2["COD_RESIDE_DEPTO"] == "47") | #Magdalena
    (df_Saber2["COD_RESIDE_DEPTO"] == "52") #Nariño
)

# Mostrar el DataFrame filtrado
df_Saber3.show()

▶ (1) Spark Jobs

▶ df_Saber3: pyspark.sql.dataframe.DataFrame = [PERIODO: string, COD_RESIDE_DEPTO: string ... 4 more fields]

+-----+-----+-----+-----+-----+-----+
|PERIODO|COD_RESIDE_DEPTO|DEPTO_RESIDE|MCPIO_RESIDE|TIENE_INTERNET|PUNT_GLOBAL|
+-----+-----+-----+-----+-----+-----+
|  20194|          47|  MAGDALENA|    SANTA ANA|             Si|        208|
|  20194|          47|  MAGDALENA|    CONCORDIA|             Si|        166|
|  20194|          47|  MAGDALENA|    SANTA MARTA|             Si|        302|
+-----+-----+-----+-----+-----+-----+

```

Por último, se revisa que los tipos de datos de las variables sean correctos:

```
#Se revisa el tipo de dato de las variables escogidas
df_Saber3.printSchema()

root
|-- PERIODO: string (nullable = true)
|-- COD_RESIDE_DEPTO: string (nullable = true)
|-- DEPTO_RESIDE: string (nullable = false)
|-- MCPIO_RESIDE: string (nullable = false)
|-- TIENE_INTERNET: string (nullable = false)
|-- PUNT_GLOBAL: string (nullable = true)
```

Se puede observar que la variable “PUNT_GLOBAL” es de tipo String, lo cual es erróneo y se procede a hacer el casteo a tipo entero (Integer):

```
#Se realiza un casteo a la variable "PUNT_GLOBAL" de string a entero:
df_Saber3 = df_Saber3.withColumn("PUNT_GLOBAL", df_Saber3["PUNT_GLOBAL"].cast(IntegerType()))

▼ df_Saber3: pyspark.sql.dataframe.DataFrame
  PERIODO: string
  COD_RESIDE_DEPTO: string
  DEPTO_RESIDE: string
  MCPIO_RESIDE: string
  TIENE_INTERNET: string
  PUNT_GLOBAL: integer
```

2. PREGUNTAS DEL NEGOCIO

De las ocho preguntas planteadas en la entrega anterior, se responderán las siguientes:

- ¿Cómo afectan los resultados académicos y la formación estudiantil, la frecuencia al acceso a internet y las diferentes herramientas tecnológicas presentes en los establecimientos académicos?
- ¿Hay una diferencia significativa en la proyección económica según los resultados académicos, entre los municipios al norte del país respecto a los elegidos en el sur?
- ¿Cuál es el índice de cobertura neta en educación de los municipios seleccionados?
- ¿Cuál es el índice de desigualdad de acceso a internet en los municipios del norte? ¿Cuál es con los municipios del sur?
- ¿Cuál es la variación en la tasa de matriculación de colegios dependiendo del desarrollo económico del municipio? Sacar conclusiones.

A continuación, se mostrará el análisis realizado para responder cada una de estas preguntas.

- ¿Cuál es el índice de desigualdad de acceso a internet en los municipios del norte? ¿Cuál es con los municipios del sur?

Esta pregunta está orientada al Dataframe de Penetración de Internet por Municipio. Para calcular la desigualdad, se utiliza un concepto interesante de las ciencias económicas y sociales y es el índice de Gini. El cual se basa en la curva de Lorenz que es una curva de frecuencia acumulada que compara la distribución de una variable específica con una distribución uniforme que pueda representar la igualdad. Normalmente, este índice se calcula con los

ingresos. Sin embargo, para calcular la desigualdad de acceso a internet entre ambas zonas del país, se realizó el análisis en base a lo siguiente:

1. Se calcula la tasa de acceso a internet, el cual representa el grado de acceso a internet en cada municipio. De esta forma, se mide cuan uniformemente se distribuye el acceso a internet entre los municipios de cada departamento. Esto se obtiene tomando las variables “No_ACCESOS_FIJOS_A_INTERNET” y “POBLACION_DANE”. Se realiza un cociente entre estas variables y se obtiene una nueva variable dentro del conjunto de datos: “tasa_acceso”.

```
#Primer paso: Se agrega una nueva columna en la tabla que guarde los datos del procedimiento para obtener la tasa de acceso a internet
df_Internet4 = df_Internet3.withColumn("tasa_acceso", col("No_ACCESOS_FIJOS_A_INTERNET") / col("POBLACION_DANE"))
df_Internet4.show()
df_Internet4.printSchema()
```

▶ (1) Spark Jobs

▶ df_Internet4: pyspark.sql.dataframe.DataFrame = [ANIO: integer, DEPARTAMENTO: string ... 4 more fields]

ANIO	DEPARTAMENTO	MUNICIPIO	No_ACCESOS_FIJOS_A_INTERNET	POBLACION_DANE	tasa_acceso
2023	NARIÑO	IPITALES	11312	120842	0.09360983763923139
2021	NARIÑO	OLAYA HERRERA	94	25969	0.003619700412029...
2022	NARIÑO	LA UNIÓN	451	33028	0.013655080537725565
2023	NARIÑO	PASTO	87688	410835	0.21343848503657187
2023	NARIÑO	SANDONÁ	680	21244	0.03200903784598004
2022	MAGDALENA	CIÉNAGA	11067	130000	0.08513076923076923
2022	NARIÑO	CHACHAGÜÍ	1877	15769	0.11903101020990552
2022	NARIÑO	LA UNIÓN	399	33028	0.01208065883492794
2022	NARIÑO	BUESACO	941	24726	0.03805710588044973
2022	MAGDALENA	EL BANCO	1776	72286	0.02456907284951443
2023	NARIÑO	POLICARPA	968	10314	0.0938530153189839
2021	NARIÑO	GUACHUCAL	309	20089	0.015381552093185325
2021	MAGDALENA	TENERIFE	822	14271	0.05759932730712634
2021	NARIÑO	ALDANA	23	7636	0.003012048192771...
2021	MAGDALENA	TENERIFE	826	14271	0.057879616004484616
2021	MAGDALENA	SALAMINA	1037	11657	0.08895942352234709
2021	NARIÑO	PUPIALES	265	17946	0.014766521787584978
2022	MAGDALENA	CERRO SAN ANTONIO	6	10954	5.477451159393829E-4

2. Teniendo esto, se procede a calcular el índice de desigualdad. El código implementa el cálculo del Coeficiente de Gini, que es una medida de desigualdad que varía entre 0 (perfecta igualdad) y 1 (máxima desigualdad). La función gini_coefficient realiza los siguientes pasos: Ordena los datos de forma ascendente, calcula el número total de observaciones (n), suma todos los valores de la columna, calcula las sumas acumulativas y aplica la fórmula del coeficiente de Gini utilizando estos valores.


```

# Segundo Paso: Crear una función para calcular el Gini
def gini_coefficient(df, column):
    # Ordenar la columna en forma ascendente
    df = df.orderBy(column)

    # Calcular el número de elementos
    n = df.count()

    # Calcular el total de la columna
    sum_yi = df.select(F.sum(column)).first()[0]

    # Calcular la suma acumulativa de los valores en la columna
    window_spec = Window.orderBy(column).rowsBetween(Window.unboundedPreceding, 0)
    df = df.withColumn("cumulative", F.sum(column).over(window_spec))

    # Obtener la suma total de los valores acumulados
    sum_cumulative = df.select(F.sum("cumulative")).first()[0]

    # Calcular el índice de Gini utilizando la fórmula correcta
    gini = (2 * sum_cumulative / (n * sum_yi)) - (n + 1) / n

    return abs(gini) # Asegurarse de que el índice sea positivo

# Calcular Gini para Nariño y Magdalena
gini_narino = gini_coefficient(df_Internet4.filter(df_Internet4.DEPARTAMENTO == "NARIÑO"), "tasa_acceso")
gini_magdalena = gini_coefficient(df_Internet4.filter(df_Internet4.DEPARTAMENTO == "MAGDALENA"), "tasa_acceso")

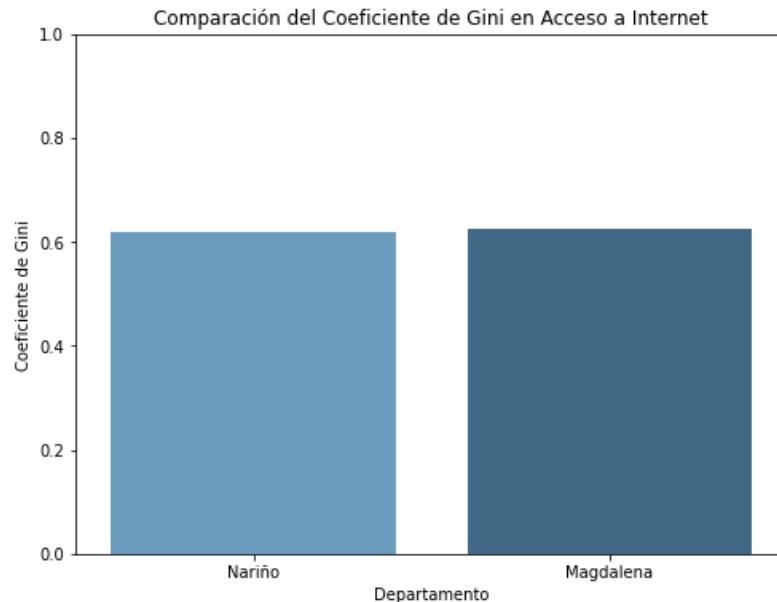
# Imprimir los resultados
print(f"Gini para Nariño: {gini_narino}")
print(f"Gini para Magdalena: {gini_magdalena}")

# Crear el gráfico comparativo de Gini
plt.figure(figsize=(8, 6))
sns.barplot(x=["Nariño", "Magdalena"], y=[gini_narino, gini_magdalena], palette="Blues_d")
plt.title("Comparación del Coeficiente de Gini en Acceso a Internet")
plt.xlabel("Departamento")
plt.ylabel("Coeficiente de Gini")
plt.ylim(0, 1) # Configurar el límite para que vaya de 0 a 1, típico para índices de Gini
plt.show()

```

- Los resultados muestran que: Nariño tiene un coeficiente de Gini de aproximadamente 0.618 y Magdalena tiene un coeficiente de Gini de aproximadamente 0.625. Lo que significa que muestran una desigualdad considerable en el acceso a internet, ya que sus valores están más cercanos a 1 que a 0. Magdalena es ligeramente mayor que Nariño. La visualización generada ayuda a entender rápidamente que ambos departamentos enfrentan desafíos similares en términos de desigualdad en el acceso a internet, con niveles de desigualdad que podrían considerarse preocupantes al estar por encima de 0.6.

Gini para Nariño: 0.6187766113301145
Gini para Magdalena: 0.6253864543930561



Por otro lado, se realizó otra aproximación para encontrar la desigualdad. El código calcula un índice de desigualdad en el acceso a internet comparando las regiones norte (Nariño) y sur (Magdalena) de Colombia. Para cada región (norte y sur): Filtra los datos por departamento, calcula los accesos a internet por cada 100 habitantes, calcula el promedio de accesos para la región, determina la diferencia absoluta entre cada municipio y el promedio y calcula el índice de desigualdad como el promedio de estas diferencias.

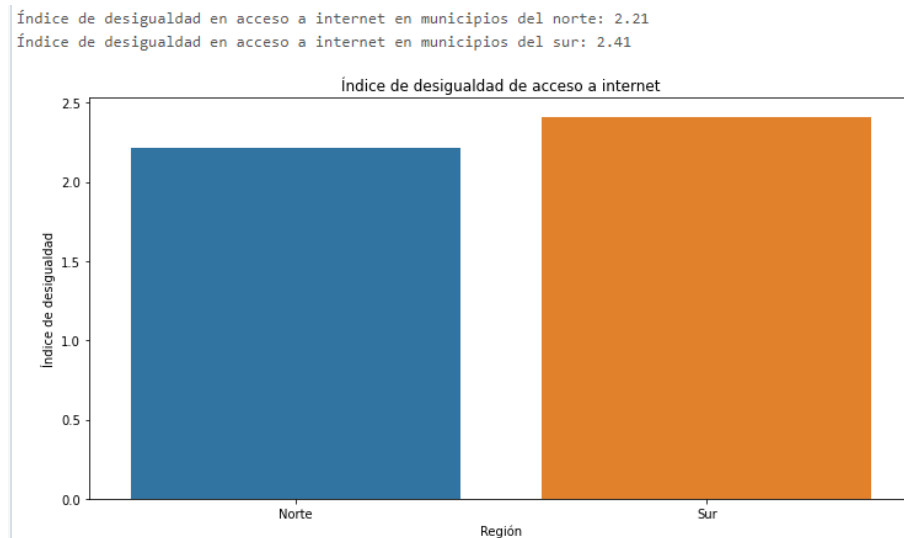
```
#Otra aproximación
# Separar los municipios del norte y del sur
norte = df_Internet4.filter(F.col("DEPARTAMENTO").isin(["NARIÑO"]))
sur = df_Internet4.filter(F.col("DEPARTAMENTO").isin(["MAGDALENA"]))

# Calcular el índice de desigualdad para el norte
norte_accesos_por_100_habitantes = (norte
    .withColumn("ACCESOS_POR_100_HABITANTES", F.round(F.col("No_ACCESOS_FIJOS_A_INTERNET") * 100.0 / F.col("POBLACION_DANE"), 2))
    .select("MUNICIPIO", "ACCESOS_POR_100_HABITANTES"))
norte_promedio = norte_accesos_por_100_habitantes.agg(F.avg("ACCESOS_POR_100_HABITANTES").alias("PROMEDIO_NORTE")).collect()[0]["PROMEDIO_NORTE"]
norte_indice_desigualdad = (norte_accesos_por_100_habitantes
    .withColumn("DIFERENCIA_PROMEDIO", F.col("ACCESOS_POR_100_HABITANTES") - norte_promedio)
    .agg(F.round(F.avg(F.abs("DIFERENCIA_PROMEDIO")), 2).alias("INDICE_DESIGUALDAD_NORTE")))
print(f"Índice de desigualdad en acceso a internet en municipios del norte: {norte_indice_desigualdad.collect()[0]['INDICE_DESIGUALDAD_NORTE']}")

# Calcular el índice de desigualdad para el sur
sur_accesos_por_100_habitantes = (sur
    .withColumn("ACCESOS_POR_100_HABITANTES", F.round(F.col("No_ACCESOS_FIJOS_A_INTERNET") * 100.0 / F.col("POBLACION_DANE"), 2))
    .select("MUNICIPIO", "ACCESOS_POR_100_HABITANTES"))
sur_promedio = sur_accesos_por_100_habitantes.agg(F.avg("ACCESOS_POR_100_HABITANTES").alias("PROMEDIO_SUR")).collect()[0]["PROMEDIO_SUR"]
sur_indice_desigualdad = (sur_accesos_por_100_habitantes
    .withColumn("DIFERENCIA_PROMEDIO", F.col("ACCESOS_POR_100_HABITANTES") - sur_promedio)
    .agg(F.round(F.avg(F.abs("DIFERENCIA_PROMEDIO")), 2).alias("INDICE_DESIGUALDAD_SUR")))
print(f"Índice de desigualdad en acceso a internet en municipios del sur: {sur_indice_desigualdad.collect()[0]['INDICE_DESIGUALDAD_SUR']}")

# Graficar los resultados
plt.figure(figsize=(12, 6))
sns.barplot(x=["Norte", "Sur"], y=[norte_indice_desigualdad.collect()[0]['INDICE_DESIGUALDAD_NORTE'],
    sur_indice_desigualdad.collect()[0]['INDICE_DESIGUALDAD_SUR']])
plt.title("Índice de desigualdad de acceso a internet")
plt.xlabel("Región")
plt.ylabel("Índice de desigualdad")
plt.show()
```

Los resultados muestran que: La región sur (Nariño) tiene un índice de desigualdad de 2.21, mientras que la región norte (Magdalena) tiene un índice de desigualdad de 2.41. La diferencia de 0.20 puntos entre ambas regiones sugiere que el sur tiene una distribución menos equitativa del acceso a internet entre sus municipios. Este índice mide qué tan lejos están en promedio los municipios del promedio regional de acceso a internet.



Esta visualización complementa el análisis anterior del coeficiente de Gini, ofreciendo otra perspectiva sobre la desigualdad en el acceso a internet en estas regiones. Mientras que el Gini mide la desigualdad en la distribución general, este índice se centra en las desviaciones promedio respecto a la media regional.

Las siguientes preguntas, se responderán en base a la información del Dataframe de Educación:

- ¿Cuál es el índice de cobertura neta en educación de los municipios seleccionados?
Se escogieron las capitales de cada departamento (Pasto y Santa Marta) y algunos municipios importantes de cada región (Plato, Arguaní, Pivijay, Ciénaga, Sitionuevo, Pupiales, San Andrés de Tumaco, Policarpa, La Cruz, Ipiales y La Unión). Se definió una lista de municipios específicos para analizar los cuales se seleccionaron de la base de datos (df_Educacion3) junto con algunas columnas relevantes para el análisis: departamento, municipio y cobertura neta para luego filtrar solo los municipios seleccionados y ordena los datos por cobertura neta en orden descendente.

```

selected_municipalities = ["Santa Marta", "Santa Ana", "Plato", "Ariguani",
                             "Pivijay", "Ciénaga", "Sitio Nuevo", "Pasto",
                             "Pupiales", "San Andres de Tumaco", "Policarpa", "La Cruz",
                             "Ipiates", "La Union"]

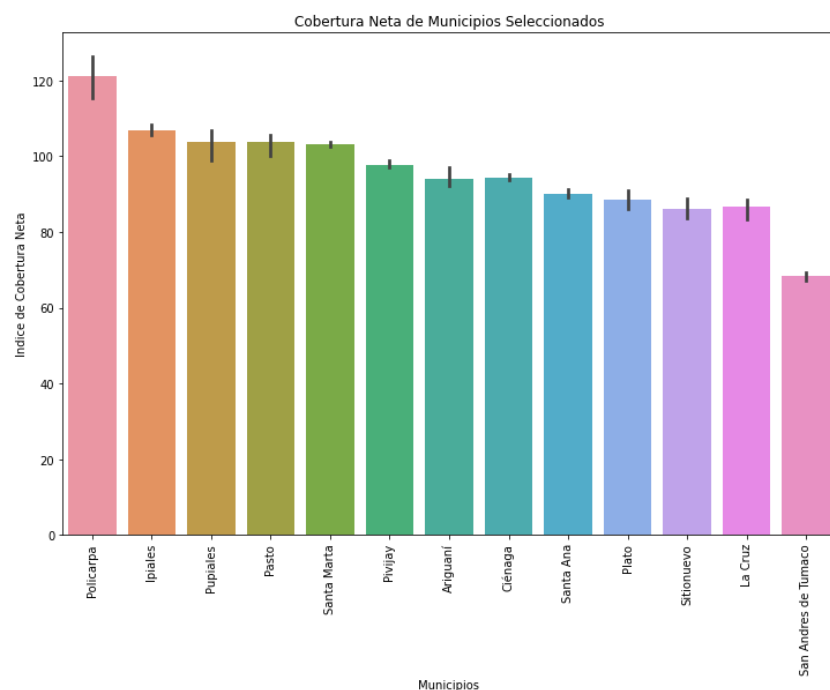
chart_df = df_Educacion3.select("DEPARTAMENTO", "MUNICIPIO", "COBERTURA_NETA") \
    .where(col("MUNICIPIO").isin(selected_municipalities)) \
    .orderBy(col("COBERTURA_NETA").desc())

# Convert Spark DataFrame to Pandas DataFrame for visualization
pd_chart_df = chart_df.toPandas()

# Create the bar chart
fig, ax = plt.subplots(figsize=(12, 8))
sns.barplot(x="MUNICIPIO", y="COBERTURA_NETA", data=pd_chart_df, ax=ax)
ax.set_title("Cobertura Neta de Municipios Seleccionados")
ax.set_xlabel("Municipios")
ax.set_ylabel("Indice de Cobertura Neta")
plt.xticks(rotation=90)
plt.show()

```

Los resultados muestran que Policarpa muestra la mayor cobertura neta, superando el 120%. La mayoría de los municipios tienen una cobertura entre 80% y 100%. San Andrés de Tumaco tiene la menor cobertura, aproximadamente 70%. Hay una gradual disminución en la cobertura entre los municipios. Lo que lleva a tener algunas observaciones: las coberturas superiores al 100% pueden indicar que hay estudiantes de otras zonas estudiando en ese municipio. Existe una brecha significativa entre los municipios con mayor y menor cobertura y la mayoría de los municipios mantienen niveles relativamente altos de cobertura.



- ¿Cuál es la variación en la tasa de matriculación de colegios dependiendo del desarrollo económico del municipio? Sacar conclusiones.

Para realizar un análisis más detallado, se seleccionaron 4 municipios de cada departamento: Santa Marta, Pasto, San Andrés de Tumaco y Plato. De esta forma, se realiza una categorización del desarrollo económico basado en la tasa de matriculación:

- Low: < 80%
- Medium: 80-90%
- High: > 90%

```
# Filtrar solo Santa Marta, Plato, San Andres de Tumaco y Pasto del DataFrame df_Educacion3
selected_cities = df_Educacion3.filter(col("MUNICIPIO").isin(["Santa Marta", "Pasto", "San Andres de Tumaco", "Plato"]))

# Categorizar el desarrollo económico
selected_cities = selected_cities.withColumn("ECONOMIC_DEVELOPMENT",
                                             when(col("TASA_M_5_16") < 80, "Low")
                                             .when((col("TASA_M_5_16") >= 80) & (col("TASA_M_5_16") < 90), "Medium")
                                             .when(col("TASA_M_5_16") >= 90, "High")
                                             .otherwise("Unknown"))

# Crear un DataFrame con las métricas relevantes
comparison_df = selected_cities.select("MUNICIPIO", "TASA_M_5_16", "COBERTURA_NETA", "ECONOMIC_DEVELOPMENT")

# Convertir a Pandas para visualización
pd_comparison = comparison_df.toPandas()

# Crear gráfico de barras comparativo
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))

# Gráfico de Tasa de Matriculación
sns.barplot(x="MUNICIPIO", y="TASA_M_5_16", data=pd_comparison, ax=ax1)
ax1.set_title("Tasa de Matriculación")
ax1.set_ylabel("Tasa (%)")

# Gráfico de Cobertura Neta
sns.barplot(x="MUNICIPIO", y="COBERTURA_NETA", data=pd_comparison, ax=ax2)
ax2.set_title("Cobertura Neta")
ax2.set_ylabel("Cobertura (%)")

plt.tight_layout()
plt.show()

# Mostrar las estadísticas detalladas
print("\nEstadísticas detalladas:")
comparison_df.show(truncate=False)
```

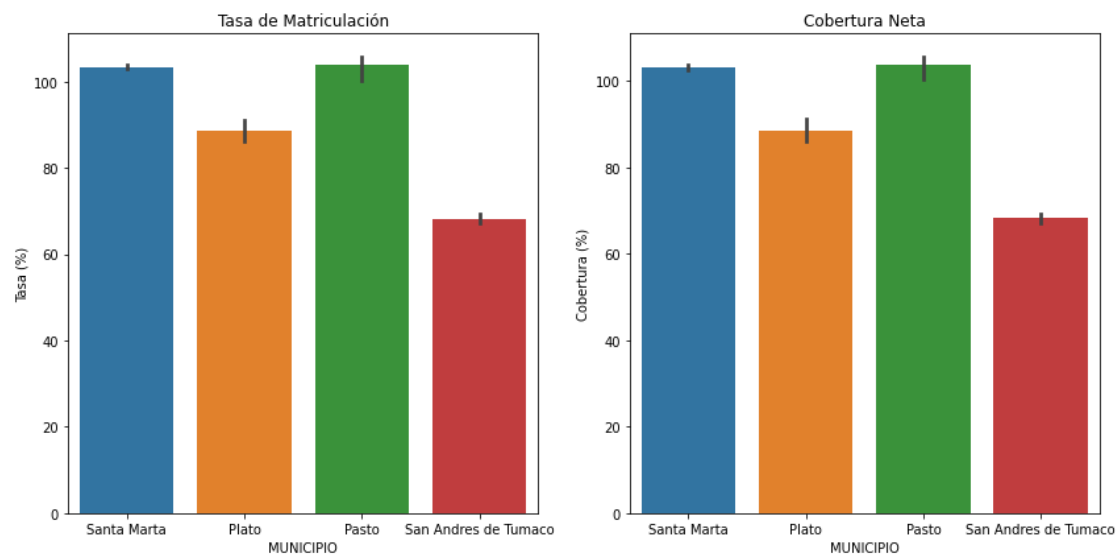
Los resultados mostraron lo siguiente:

- Gráfico de Tasa de Matriculación
 - Muestra el porcentaje de estudiantes matriculados en cada municipio.
 - Santa Marte y Pasto superan el 100% mientras que Plato se encuentra cerca del 90% y San Andrés de Tumaco, este alrededor del 70%.
- Gráfico de Cobertura Neta
 - Muestra resultados muy similares a la Tasa de Matriculación. Mantiene el mismo patrón entre municipios por lo que se confirman que hay brechas educativas entres estas regiones.
- Tabla de Estadísticas:

- Recopila la información de las gráficas e incluye la clasificación antes mencionada.
- Revela patrones consistentes a lo largo del tiempo. Santa Marta y Pasto muestran consistentemente desarrollo "High". San Andrés de Tumaco se mantiene en "Low". Plato fluctúa entre "Medium" y "High"

Estos resultados muestran una serie de hallazgos interesantes: Existe una clara correlación entre tasa de matriculación y cobertura neta se cuenta con una brecha significativa entre los municipios de mejor y peor desempeño. Santa Marta y Pasto muestran indicadores superiores al 100% mientras que San Andrés de Tumaco muestra consistentemente los indicadores más bajos

Estos resultados generan una serie de implicaciones: Se evidencian desigualdades significativas en el acceso a la educación ya que los municipios con mejor desarrollo económico tienden a tener mejores indicadores educativos. Sin embargo, San Andres de Tumaco al ser marginado como una zona de peligro a tenido índices económicos diferentes frente al turismo por lo que puede ser una de las razones por la cual, al ser un destino turístico, presenta diferentes índices económicos frente a otros municipios como lo es Pasto y Santa Marta.



Estadísticas detalladas:

MUNICIPIO	TASA_M_5_16	COBERTURA_NETA	ECONOMIC_DEVELOPMENT
Santa Marta	102.9	102.57	High
Plato	91.18	91.13	High
Pasto	105.24	105.07	High
San Andres de Tumaco	67.18	67.16	Low
Santa Marta	103.33	102.97	High
Plato	88.36	88.36	Medium
Pasto	105.89	105.66	High
San Andres de Tumaco	68.47	68.46	Low
Santa Marta	104.08	103.7	High
Plato	86.14	86.12	Medium
Pasto	100.41	100.24	High
San Andres de Tumaco	69.24	69.22	Low

- ¿Hay una diferencia significativa en la proyección económica según los resultados académicos, entre los municipios al norte del país respecto a los elegidos en el sur?

Esta pregunta se responde con el Dataframe de ICFES Saber 11.

Antes de iniciar el análisis, se realiza una segunda limpieza de la variable “TIENE_INTERNET” donde se unifican los valores nulos en la columna, reemplazándolos con "No Reporta". Se separan los datos en dos regiones: Norte (Magdalena) y Sur (Nariño). Se calcula las estadísticas de cada región: se hace una agrupación (count) de la variable “PUNT_GLOBAL” y se llama como “total_estudiantes”. Se calcula el promedio (“promedio”) y se encuentra la desviación estándar (“desviación_estandar”). Se realiza el procedimiento entre ambas regiones.

```

# Limpieza de datos: unificar valores nulos
df_Saber4 = df_Saber3
df_clean = df_Saber4.withColumn(
    "TIENE_INTERNET",
    when(col("TIENE_INTERNET").isin([None, "null", "-"]), "No Reporta")
    .otherwise(col("TIENE_INTERNET"))
)

# Separar municipios por región
north_results = df_clean.filter(col("DEPTO_RESIDE") == "MAGDALENA")
south_results = df_clean.filter(col("DEPTO_RESIDE") == "NARIÑO")

def calculate_stats(df, region_name):
    return df.agg(
        count("PUNT_GLOBAL").alias("total_estudiantes"),
        avg("PUNT_GLOBAL").alias("promedio"),
        stddev("PUNT_GLOBAL").alias("desviacion_estandar")
    ).withColumn("region", lit(region_name))

# Calcular estadísticas por región
north_stats = calculate_stats(north_results, "Norte")
south_stats = calculate_stats(south_results, "Sur")
stats_comparison = north_stats.union(south_stats)

# Análisis por acceso a internet con tratamiento de nulos
internet_analysis = df_clean.groupBy("DEPTO_RESIDE", "TIENE_INTERNET") \
    .agg(
        avg("PUNT_GLOBAL").alias("promedio_puntaje"),
        count("*").alias("cantidad_estudiantes")
    ) \
    .orderBy("DEPTO_RESIDE", "TIENE_INTERNET")

```

Adicionalmente, se realizan dos análisis complementarios sobre el acceso a internet en los datos educativos. En la primera parte (internet_analysis), agrupa los datos por departamento de residencia y acceso a internet, calculando para cada grupo el promedio de puntaje global y el número de estudiantes, ordenando los resultados por departamento y estado de acceso a internet. En la segunda parte (missing_data_analysis), se enfoca en analizar la calidad de los datos, calculando qué proporción de estudiantes no reportaron su acceso a internet en cada departamento - esto se hace contando cuántos registros tienen "No Reporta" en la columna TIENE_INTERNET y dividiendo este número por el total de registros en cada departamento, expresándolo como un porcentaje. Este análisis es crucial para entender no solo la relación entre acceso a internet y rendimiento académico, sino también para evaluar la completitud y confiabilidad de los datos recolectados.


```

# Análisis adicional: proporción de datos faltantes
missing_data_analysis = df_clean.groupby("DEPTO_RESIDE") \
    .agg(
        count(when(col("TIENE_INTERNET") == "No Reporta", True)).alias("datos_no_reportados"),
        count("*").alias("total_registros")
    ) \
    .withColumn(
        "porcentaje_no_reportado",
        (col("datos_no_reportados") * 100 / col("total_registros"))
    )

# Convertir a Pandas para visualización
pd_stats = stats_comparison.toPandas()
pd_internet = internet_analysis.toPandas()
pd_missing = missing_data_analysis.toPandas()

# Crear visualizaciones
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(20, 6))

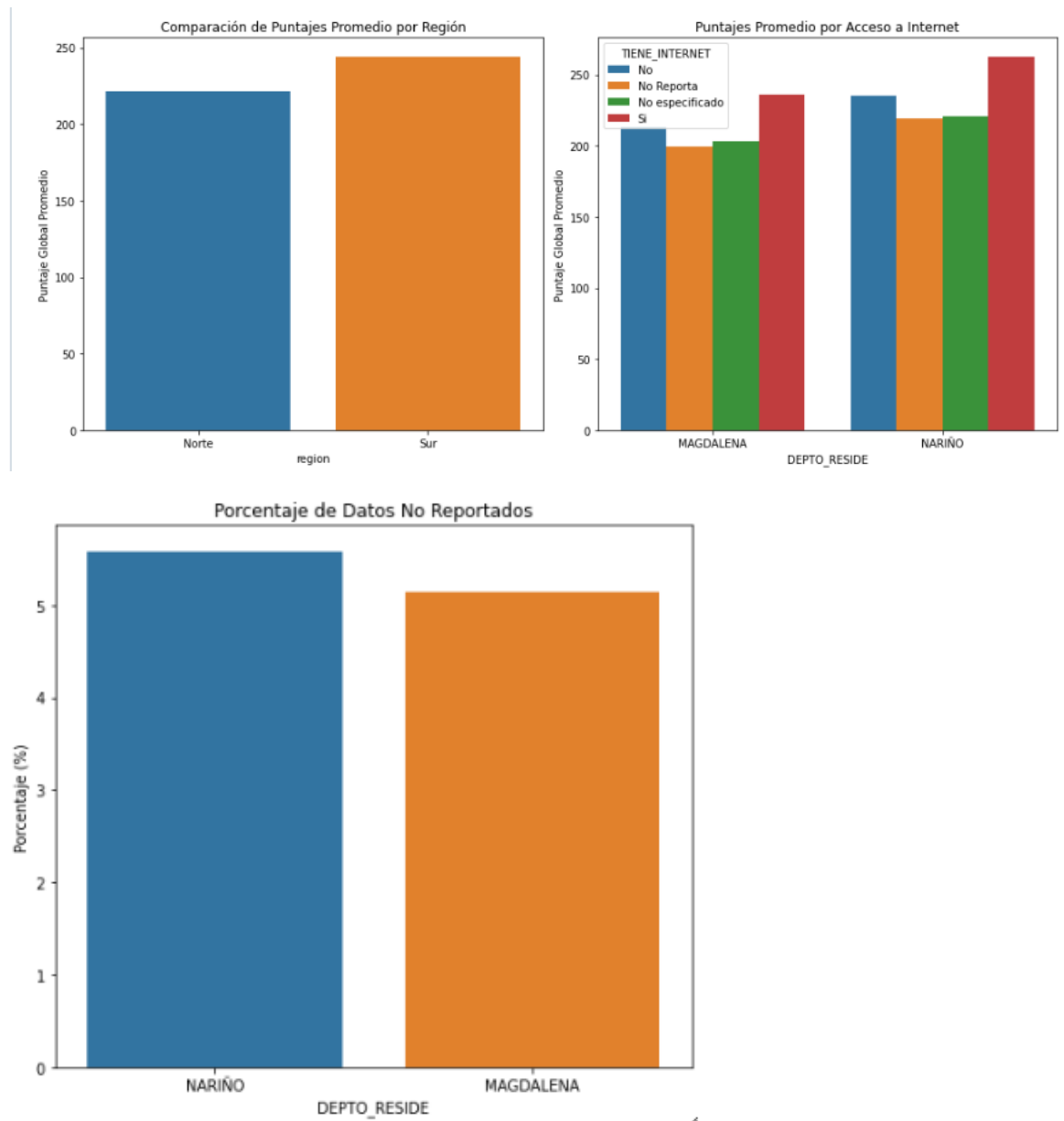
# Gráfico 1: Comparación de puntajes promedio
sns.barplot(x="region", y="promedio", data=pd_stats, ax=ax1)
ax1.set_title("Comparación de Puntajes Promedio por Región")
ax1.set_ylabel("Puntaje Global Promedio")

# Gráfico 2: Puntajes por acceso a internet
sns.barplot(x="DEPTO_RESIDE", y="promedio_puntaje", hue="TIENE_INTERNET", data=pd_internet, ax=ax2)
ax2.set_title("Puntajes Promedio por Acceso a Internet")
ax2.set_ylabel("Puntaje Global Promedio")

# Gráfico 3: Proporción de datos faltantes
sns.barplot(x="DEPTO_RESIDE", y="porcentaje_no_reportado", data=pd_missing, ax=ax3)
ax3.set_title("Porcentaje de Datos No Reportados")
ax3.set_ylabel("Porcentaje (%)")

```

Los tres gráficos revelan patrones importantes sobre el rendimiento académico y su relación con el acceso a internet en los departamentos de Magdalena y Nariño. El primer gráfico muestra una clara diferencia en el rendimiento académico entre las regiones, donde Nariño (Sur) presenta un puntaje promedio significativamente más alto que Magdalena (Norte), con una diferencia aproximada de 30-40 puntos. El segundo gráfico profundiza en esta diferencia al desglosar los puntajes según el acceso a internet, revelando un patrón consistente en ambos departamentos: los estudiantes que tienen acceso a internet ("Sí") obtienen puntajes notablemente más altos (cerca de 260 puntos en Nariño y 236 en Magdalena) que aquellos que no tienen acceso o no reportan esta información. Es particularmente interesante observar que esta tendencia se mantiene en ambos departamentos, aunque Nariño mantiene consistentemente puntajes más altos en todas las categorías. Finalmente, el tercer gráfico muestra que ambos departamentos tienen un porcentaje similar de datos no reportados (aproximadamente 5.5%), lo que sugiere una consistencia en los patrones de reporte y añade credibilidad a las comparaciones realizadas. Este análisis sugiere que tanto la ubicación geográfica como el acceso a internet son factores significativos en el rendimiento académico de los estudiantes.



Las tablas presentadas en los datos, se observa un desglose detallado que complementa y confirma lo visualizado en los gráficos. En Magdalena, de un total de 16,627 estudiantes, el acceso a internet se distribuye de la siguiente manera: 8,736 estudiantes no tienen internet (promedio de 212.80 puntos), 854 no reportan la información (199.17 puntos), 315 tienen datos no especificados (203.02 puntos), y 6,722 sí tienen acceso a internet (236.08 puntos). Por su parte, Nariño, con 16,903 estudiantes totales, muestra una distribución similar, pero con mejores promedios: 9,432 estudiantes sin internet (235.56 puntos), 944 no reportan (219.41 puntos), 373 no especificados (221.05 puntos), y 6,154 con acceso a internet (262.94 puntos). Estos datos numéricos revelan que, aunque la distribución de acceso a internet es similar en ambos departamentos, Nariño mantiene consistentemente

puntajes más altos en todas las categorías, con una diferencia particularmente notable en los estudiantes que sí tienen acceso a internet. Además, el porcentaje de datos no reportados es similar en ambos departamentos (5.13% en Magdalena y 5.58% en Nariño), lo que sugiere una consistencia en la calidad de la recolección de datos entre ambas regiones.

DEPTO_RESIDE	TIENE_INTERNET	promedio_puntaje	cantidad_estudiantes
MAGDALENA	No	212.80460164835165	8736
MAGDALENA	No Reporta	199.1791569086651	854
MAGDALENA	No especificado	203.02222222222222	315
MAGDALENA	Si	236.081523356144	6722
NARIÑO	No	235.56318914334182	9432
NARIÑO	No Reporta	219.41949152542372	944
NARIÑO	No especificado	221.05630026809652	373
NARIÑO	Si	262.9470263243419	6154

Análisis de Datos No Reportados:

DEPTO_RESIDE	datos_no_reportados	total_registros	porcentaje_no_reportado
NARIÑO	944	16903	5.5848074306336155
MAGDALENA	854	16627	5.136224213628435

- ¿Cómo afectan los resultados académicos y la formación estudiantil, la frecuencia al acceso a internet y las diferentes herramientas tecnológicas presentes en los establecimientos académicos?

Este código realiza una limpieza, análisis y visualización de datos sobre acceso a internet y rendimiento académico. Primero, estandariza los valores nulos de la columna TIENE_INTERNET en el conjunto de datos df_Saber, reemplazando valores nulos y vacíos con "No Reporta". Luego, agrupa los datos por departamento y acceso a internet para calcular el puntaje promedio global y el número de estudiantes por grupo. También analiza la cobertura educativa y tasa de matrícula en df_Educacion. Finalmente, calcula la diferencia en puntajes entre estudiantes con y sin acceso a internet.

```

# 1. Limpieza y preparación de datos
# Estandarizar valores nulos en df_Saber
df_Saber4_clean = df_Saber4.withColumn(
    "TIENE_INTERNET",
    when(col("TIENE_INTERNET").isin([None, "null", "-"]), "No Reporta")
    .otherwise(col("TIENE_INTERNET"))
)

# 2. Análisis de resultados académicos por acceso a internet
internet_analysis = df_Saber4_clean.groupBy("DEPTO_RESIDE", "TIENE_INTERNET") \
    .agg(
        round(avg("PUNT_GLOBAL"), 2).alias("promedio_puntaje"),
        count("*").alias("cantidad_estudiantes")
    ) \
    .orderBy("DEPTO_RESIDE", "TIENE_INTERNET")

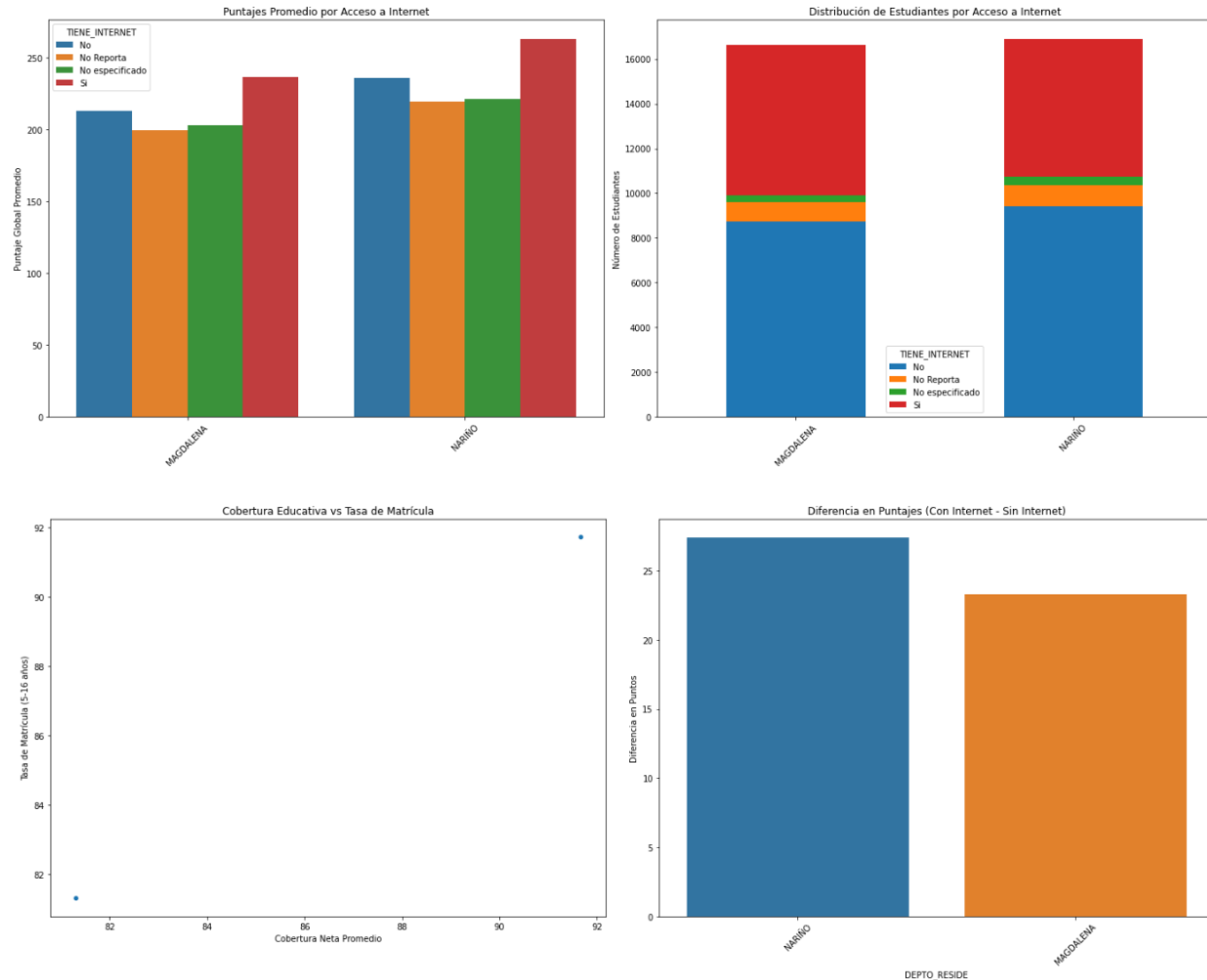
# 3. Análisis de cobertura educativa y resultados
education_metrics = df_Educacion3.groupBy("DEPARTAMENTO") \
    .agg(
        round(avg("COBERTURA_NETA"), 2).alias("cobertura_promedio"),
        round(avg("TASA_M_5_16"), 2).alias("tasa_matricula_promedio")
    )

# 4. Análisis combinado
# Calculamos la diferencia de puntaje entre estudiantes con y sin internet
internet_impact = df_Saber4_clean \
    .filter(col("TIENE_INTERNET").isin(["Si", "No"])) \
    .groupBy("DEPTO_RESIDE") \
    .pivot("TIENE_INTERNET") \
    .agg(avg("PUNT_GLOBAL")) \
    .withColumn("diferencia_puntaje", col("Si") - col("No"))

# 5. Visualizaciones
# Convertir a Pandas para visualización
pd_internet = internet_analysis.toPandas()
pd_education = education_metrics.toPandas()
pd_impact = internet_impact.toPandas()

```

Los gráficos muestran distintos aspectos del análisis. El primero compara puntajes promedio por acceso a internet, destacando variaciones en puntajes por departamento y acceso. El segundo gráfico ilustra la distribución de estudiantes según su acceso a internet, revelando la proporción de estudiantes en cada categoría. El tercer gráfico es un diagrama de dispersión que relaciona cobertura educativa con la tasa de matrícula. El cuarto gráfico muestra la diferencia en puntajes entre estudiantes con y sin internet en distintos departamentos.



En la tabla **Estadísticas por Acceso a Internet**, se observan los puntajes promedio y la cantidad de estudiantes en cada categoría de acceso a internet por departamento. En Magdalena, los estudiantes sin acceso a internet obtuvieron un puntaje promedio de 212.8, mientras que aquellos con acceso alcanzaron 236.08. En Nariño, el promedio de los estudiantes sin internet fue de 235.56, en contraste con los 262.95 de aquellos con acceso. Esto muestra que los estudiantes con internet tienden a obtener puntajes más altos en ambos departamentos.

La tabla **Métricas Educativas por Departamento** muestra la cobertura promedio y la tasa de matrícula promedio en Magdalena y Nariño. Magdalena presenta una cobertura y tasa de matrícula del 91.66% y 91.73%, respectivamente, mientras que Nariño tiene valores más bajos, con un 81.3% de cobertura y una tasa de matrícula del 81.31%. Esto sugiere una mayor accesibilidad a la educación en Magdalena comparado con Nariño.

Finalmente, la tabla **Impacto del Internet en Puntajes** destaca la diferencia en el puntaje promedio entre estudiantes con y sin internet por departamento. En Nariño, la diferencia es de 27.38 puntos a favor de quienes tienen internet, mientras que en Magdalena la

diferencia es de 23.28 puntos. Esto refuerza la observación de que el acceso a internet parece tener un impacto positivo en los puntajes académicos.

Estadísticas por Acceso a Internet:

DEPTO_RESIDE	TIENE_INTERNET	promedio_puntaje	cantidad_estudiantes
MAGDALENA	No	212.8	8736
MAGDALENA	No Reporta	199.18	854
MAGDALENA	No especificado	203.02	315
MAGDALENA	Si	236.08	6722
NARIÑO	No	235.56	9432
NARIÑO	No Reporta	219.42	944
NARIÑO	No especificado	221.06	373
NARIÑO	Si	262.95	6154

Métricas Educativas por Departamento:

DEPARTAMENTO	cobertura_promedio	tasa_matricula_promedio
Magdalena	91.66	91.73
Nariño	81.3	81.31

Impacto del Internet en Puntajes:

DEPTO_RESIDE	No	Si	diferencia_puntaje
NARIÑO	235.56318914334182	262.9470263243419	27.383837181000075
MAGDALENA	212.80460164835165	236.081523356144	23.276921707792354

3. APRENDIZAJE DE MAQUINA

Los algoritmos de **aprendizaje supervisado** y **aprendizaje no supervisado** son dos enfoques distintos en el aprendizaje automático, diferenciados principalmente por el tipo de datos y objetivos que tienen.

- **Aprendizaje supervisado:** En este enfoque, el algoritmo aprende a partir de un conjunto de datos etiquetado, es decir, donde cada entrada está asociada con una etiqueta o valor objetivo conocido. Esto significa que el modelo sabe cuál es la "respuesta correcta" mientras entrena, y su tarea es aprender la relación entre las características de entrada y esa respuesta. Los algoritmos supervisados, como **Regresión Lineal** y **Random Forest**, se utilizan comúnmente para tareas de predicción y clasificación, como predecir el precio de una casa o clasificar correos electrónicos

como "spam" o "no spam". La clave es que el modelo tiene una "guía" durante el entrenamiento, lo cual permite medir su precisión y ajustar su rendimiento de manera explícita.

- **Aprendizaje no supervisado:** En este enfoque, el modelo trabaja con datos sin etiquetas ni respuestas conocidas. Aquí, el objetivo es descubrir patrones o estructuras ocultas en los datos, sin ninguna instrucción específica sobre qué buscar. Los algoritmos no supervisados, como **K-means** y **Análisis de Componentes Principales (PCA)**, son utilizados para agrupación, reducción de dimensionalidad y análisis exploratorio. Por ejemplo, K-means puede agrupar clientes con características similares en "clusters" para identificar diferentes segmentos de mercado. La falta de etiquetas implica que no hay una "respuesta correcta" en estos algoritmos, y el modelo debe encontrar la estructura de los datos de manera autónoma.

Random Forest se usa para solucionar problemas de clasificación y regresión. Cada árbol genera una predicción, y la decisión final se toma a través del voto mayoritario (en clasificación) o el promedio (en regresión) de todos los árboles. La aleatoriedad en el proceso reduce el riesgo de sobreajuste y mejora el desempeño general del modelo, especialmente en conjuntos de datos complejos y de alta dimensionalidad.

El algoritmo **K-means** es una técnica de aprendizaje no supervisado que agrupa los datos en un número especificado de clusters o grupos, representados por el valor de K . Su objetivo es dividir los datos en clusters de manera que los puntos dentro de cada grupo sean lo más similares posible entre sí y lo más diferentes a los puntos de otros grupos. El proceso inicia asignando aleatoriamente K centroides, uno para cada cluster, y luego asignando cada punto al centroide más cercano. Posteriormente, los centroides se recalculan como el promedio de los puntos en cada grupo, y el proceso se repite hasta que los centroides se estabilizan. K-means es útil para identificar patrones en datos no etiquetados, como segmentación de clientes o agrupación de regiones con características similares.

Este código aplica técnicas de clasificación y clustering para analizar la relación entre el acceso a Internet y el rendimiento académico en diferentes municipios, usando PySpark para la manipulación de datos y el entrenamiento de modelos. La metodología combina aprendizaje supervisado, y aprendizaje no supervisado, como anteriormente se explicó, para explorar patrones y segmentar los datos en función de variables clave.

La función `prepare_data_for_rf` limpia y transforma los datos para el modelo de Random Forest. Filtra el conjunto de datos `df_Saber4`, eliminando valores nulos en las columnas `TIENE_INTERNET` y `PUNT_GLOBAL`, y convierte `TIENE_INTERNET` en una variable numérica (1 si tiene acceso, 0 si no). Luego, se realiza una unión con el dataset `df_Internet4` en base a municipio y departamento, para integrar datos de acceso a Internet. Finalmente, se crea una variable binaria `high_performance` que clasifica a los estudiantes en función de si su puntaje (`PUNT_GLOBAL`) está por encima o por debajo de la mediana, y se ensamblan las características para entrenar el modelo.

La función `train_random_forest` entrena el modelo de Random Forest. Primero divide los datos en conjuntos de entrenamiento y prueba en una proporción de 80%-20%, entrena el modelo con 50 árboles y una profundidad máxima de 5 para mejorar la eficiencia, y evalúa su precisión usando `MulticlassClassificationEvaluator`. Esta evaluación calcula qué tan bien el modelo clasifica el rendimiento académico de los estudiantes en función del acceso a Internet y la tasa de acceso.

```
def prepare_data_for_rf(df_Internet4, df_Saber4):
    """Prepara los datos para Random Forest manejando valores nulos"""
    # Limpiamos los datos primero
    df_Saber4_clean = df_Saber4.filter(
        (F.col("TIENE_INTERNET").isNotNull()) &
        (F.col("PUNT_GLOBAL").isNotNull()) &
        (F.col("TIENE_INTERNET") != "-")
    )

    # Convertir TIENE_INTERNET a numérico directamente
    df_Saber4_clean = df_Saber4_clean.withColumn(
        "internet_numeric",
        F.when(F.col("TIENE_INTERNET") == "Si", 1.0)
        .otherwise(0.0)
    )

    # Unir los dataframes por municipio
    df_joined = df_Saber4_clean.join(
        df_Internet4,
        (df_Saber4_clean.MCPIO_RESIDE == df_Internet4.MUNICIPIO) &
        (df_Saber4_clean.DEPTO_RESIDE == df_Internet4.DEPARTAMENTO),
        'inner' # Cambiamos a inner join para asegurar correspondencia
    )

    # Crear variable objetivo binaria (alto/bajo rendimiento)
    median_score = df_joined.approxQuantile("PUNT_GLOBAL", [0.5], 0.01)[0]
    df_joined = df_joined.withColumn(
        "high_performance",
        F.when(F.col("PUNT_GLOBAL") > median_score, 1.0).otherwise(0.0)
    )

    # Preparar features sin usar StringIndexer
    feature_cols = ["internet_numeric", "tasa_acceso"]
    assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")
    final_data = assembler.transform(df_joined)

    return final_data.select("features", "high_performance", "PUNT_GLOBAL")
```

```
def train_random_forest(prepared_data):
    """Entrena el modelo Random Forest"""
    # Dividir datos
    train_data, test_data = prepared_data.randomSplit([0.8, 0.2], seed=42)

    # Entrenar modelo
    rf = RandomForestClassifier(
        labelCol="high_performance",
        featuresCol="features",
        numTrees=50, # Reducimos el número de árboles para mayor eficiencia
        maxDepth=5
    )
    model = rf.fit(train_data)

    # Evaluar modelo
    predictions = model.transform(test_data)
    evaluator = MulticlassClassificationEvaluator(
        labelCol="high_performance",
        predictionCol="prediction",
        metricName="accuracy"
    )
    accuracy = evaluator.evaluate(predictions)

    return model, predictions, accuracy
```

La función `prepare_data_for_kmeans` prepara los datos para el modelo K-means, calculando estadísticas agregadas por municipio, como el puntaje promedio y el número de estudiantes. Luego, estas estadísticas se unen con el dataset de acceso a Internet (`df_Internet4`), y se normalizan las características `avg_score` y `tasa_acceso` para facilitar la segmentación. Finalmente, se ensamblan estas columnas normalizadas en un vector de características que se usará para el clustering.

Para visualizar los resultados, la función `create_visualizations` genera gráficos que muestran la importancia de las características en el modelo Random Forest, la distribución de puntajes según las predicciones del modelo, la distribución de clusters obtenidos por K-means y los atributos de cada cluster. Estas visualizaciones permiten interpretar visualmente los resultados y facilitan la comparación entre los municipios según sus características de acceso a Internet y rendimiento académico.


```
def prepare_data_for_kmeans(df_Internet4, df_Saber4):
    """Prepara los datos para K-means"""
    # Limpiamos los datos primero
    df_Saber4_clean = df_Saber4.filter(
        F.col("PUNT_GLOBAL").isNotNull()
    )

    # Agregar estadísticas por municipio
    stats_by_municipality = df_Saber4_clean.groupBy(
        "MCPIO_RESIDE", "DEPTO_RESIDE"
    ).agg(
        F.avg("PUNT_GLOBAL").alias("avg_score"),
        F.count("*").alias("student_count")
    )

    # Unir con datos de internet
    df_merged = stats_by_municipality.join(
        df_Internet4,
        (stats_by_municipality.MCPIO_RESIDE == df_Internet4.MUNICIPIO) &
        (stats_by_municipality.DEPTO_RESIDE == df_Internet4.DEPARTAMENTO),
        'inner'
    )

    # Crear ventana para todo el dataset
    windowSpec = Window.partitionBy()

    # Normalizar las características usando la ventana
    df_merged = df_merged.withColumn(
        "normalized_score",
        (F.col("avg_score") - F.min("avg_score").over(windowSpec)) /
        (F.max("avg_score").over(windowSpec) - F.min("avg_score").over(windowSpec))
    ).withColumn(
        "normalized_access",
        F.col("tasa_acceso") # Ya está en proporción
    )

    # Normalizar las características usando la ventana
    df_merged = df_merged.withColumn(
        "normalized_score",
        (F.col("avg_score") - F.min("avg_score").over(windowSpec)) /
        (F.max("avg_score").over(windowSpec) - F.min("avg_score").over(windowSpec))
    ).withColumn(
        "normalized_access",
        F.col("tasa_acceso") # Ya está en proporción
    )

    # Preparar features para clustering
    assembler = VectorAssembler(
        inputCols=["normalized_score", "normalized_access"],
        outputCol="features"
    )

    return assembler.transform(df_merged)

def create_visualizations(rf_model, rf_predictions, kmeans_model, cluster_predictions):
    """Crea visualizaciones para ambos modelos"""
    plt.style.use('seaborn')
    fig = plt.figure(figsize=(20, 15))

    # 1. Importancia de características (Random Forest)
    ax1 = plt.subplot(2, 2, 1)
    feature_importance = pd.DataFrame({
        'feature': ['Acceso Internet', 'Tasa Acceso'],
        'importance': rf_model.featureImportances.toArray()
    })
    sns.barplot(
        data=feature_importance,
        x='feature',
        y='importance',
        ax=ax1
    )
    ax1.set_title('Importancia de Variables en Random Forest')
    ax1.set_xticklabels(ax1.get_xticklabels(), rotation=45)
```

Finalmente, la función `print_analysis` imprime un análisis detallado de los resultados, incluyendo la precisión del modelo Random Forest y las características de cada cluster creado. El análisis ayuda a entender los patrones encontrados, como el rendimiento académico promedio y la tasa de acceso en cada grupo de municipios.

La función `run_complete_analysis` ejecuta todo el flujo de trabajo, llamando a cada función en el orden adecuado. Esto permite preparar los datos, entrenar ambos modelos, generar visualizaciones y realizar un análisis completo en un solo paso. Al final, se muestran los gráficos generados, proporcionando una visión comprensiva del impacto del acceso a Internet en el rendimiento académico a nivel de municipio y cómo los municipios se agrupan en función de estas variables.

```

def print_analysis(rf_accuracy, cluster_predictions):
    """Imprime análisis detallado de los resultados"""
    print("=== Resultados del Análisis ===")
    print(f"\n1. Random Forest:")
    print(f"Precisión del modelo: {rf_accuracy:.2%}")

    print("\n2. Análisis de Clusters:")
    cluster_predictions.groupby('prediction').agg(
        F.count('*').alias('cantidad_municipios'),
        F.avg('avg_score').alias('puntaje_promedio'),
        F.avg('tasa_acceso').alias('tasa_acceso_promedio')
    ).orderBy('prediction').show()

def run_complete_analysis(df_Internet4, df_Saber4):
    """Ejecuta el análisis completo"""
    print("Preparando datos para Random Forest...")
    rf_data = prepare_data_for_rf(df_Internet4, df_Saber4)

    print("Entrenando Random Forest...")
    rf_model, rf_predictions, rf_accuracy = train_random_forest(rf_data)

    print("Preparando datos para K-means...")
    kmeans_data = prepare_data_for_kmeans(df_Internet4, df_Saber4)

    print("Entrenando K-means...")
    kmeans = KMeans(k=3, seed=42)
    kmeans_model = kmeans.fit(kmeans_data)
    cluster_predictions = kmeans_model.transform(kmeans_data)

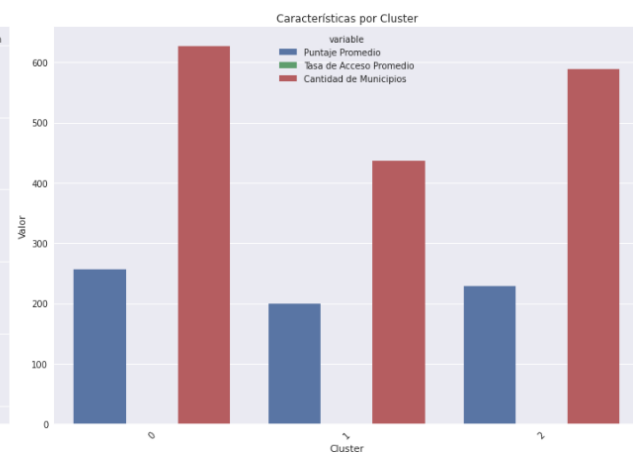
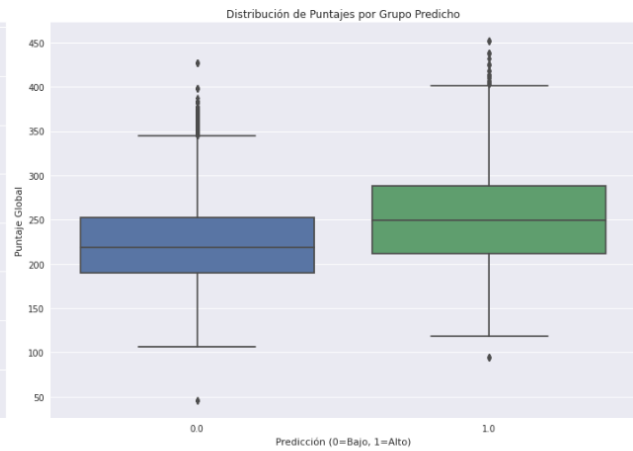
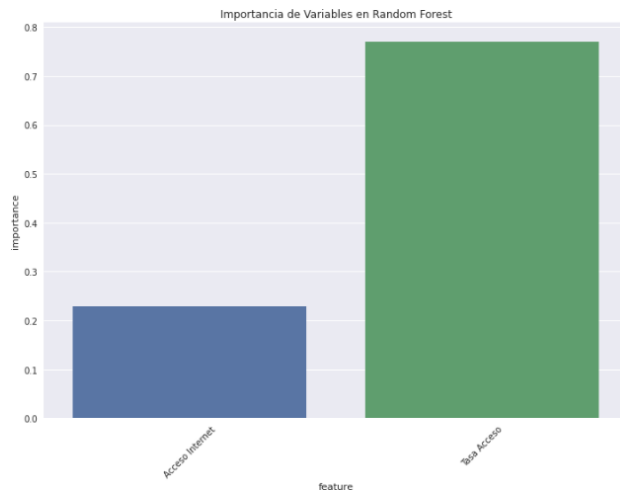
    print("Creando visualizaciones...")
    fig = create_visualizations(rf_model, rf_predictions, kmeans_model, cluster_predictions)

    print("Generando análisis...")
    print_analysis(rf_accuracy, cluster_predictions)

    return fig, rf_model, kmeans_model

```

Los gráficos presentan un análisis complementario utilizando técnicas de aprendizaje supervisado (Random Forest) y no supervisado (K-means) para estudiar la relación entre el acceso a internet y ciertos puntajes en diferentes municipios. El Random Forest revela que la tasa de acceso es la variable más influyente en las predicciones, con una importancia relativa cercana al 75%, mientras que el acceso a internet contribuye en aproximadamente un 22%. Esta diferencia se refleja en la distribución de puntajes, donde se observa una clara separación entre grupos altos y bajos. Por su parte, el análisis de K-means identifica tres clusters distintos de municipios: un grupo mayoritario con puntajes intermedios, otro con puntajes más bajos y un tercer grupo más pequeño, pero con puntajes superiores. La visualización de clusters muestra una segregación clara en función de la tasa de acceso a internet normalizada y los puntajes promedios, sugiriendo que existe una relación estructural entre el acceso a tecnología y los resultados obtenidos en los municipios analizados. Ambos métodos convergen en señalar la importancia del acceso tecnológico como factor diferenciador entre los distintos grupos de municipios.



4. PREPARACIÓN DE DATOS PARA EL MODELADO

5. TECNICAS CON MLIB

6. METRICAS

7. CONCLUSIONES

8. REFERENCIAS

- Sanz, F. (2020, noviembre 26). *Algoritmo K-Means Clustering – aplicaciones y desventajas*. The Machine Learners. <https://www.themachinelearners.com/k-means/>
- ¿Qué es Random Forest? (2023, mayo 3). *Ibm.com*. <https://www.ibm.com/mx-es/topics/random-forest>
- Martínez, S. (2023, enero 27). *Random forest, la gran técnica de Machine Learning*. Inesdi. <https://www.inesdi.com/blog/random-forest-que-es/>